Lab 5: USB

Student: Ilya-Linh Nguyen

Innopolis main: i.nguen@innopolis.university

Group: CBS-01

Task

- Implement Lab 4 first (it will be used for further improvement).
- Add any USB device as an electronic key for your chardev (from lab 4). You can use any VID/PID: mouse, keyboard, usb stick, etc.
- Chardev (from lab 4) must not appear in the system unless electronic key is not inserted into USB port.
- In case of usb device removal (from lab 4) chardev must be also removed from /dev list but stack must not be destroyed.
- Add `error: USB key not inserted` to your userspace wrapper (kernel_stack) from lab 4.

Solution

Here you can see the workflow of the lab and files:

- Asciinema URL: https://asciinema.org/a/yyF0571NJa4Ztka5
- GitHub URL: https://github.com/ilyalinhnguyen/adv-linux

I did not do screenshots in this lab because Asciinema will show the whole workflow of the program.

Kernel module: USB-gated chardev

The module registers a USB driver and a character device:

- **Always present in memory**: the stack buffer (`data/capacity/top`) exists regardless of USB presence.
- **Appears in `/dev` only with key**: when the USB key is inserted, the module calls `device_create(...)` and `/dev/int_stack` becomes available.
- **Disappears on removal**: when the USB key is removed, the module calls `device_destroy(...)` so `/dev/int_stack` is removed, but the stack content is preserved in RAM.

Choosing the USB key (VID/PID)

The module uses a USB notifier and a **hardcoded** VID/PID for the key:

- `0951:1666` (Kingston DataTraveler 100 G3/G4/SE9 G2/50 Kyson) — **this is my Ventoy USB device**

It does not “bind” to the device, so it does not interfere with the normal `usb-storage` driver.

To pick a different USB key, find the VID/PID via:

```
lsusb
```

Then update `USB_KEY_VID` / `USB_KEY_PID` in `lab5/usb_int_stack.c` and rebuild.

Notes on implementation

- The USB notifier reacts to add/remove events; the key is selected by the hardcoded VID/PID.
- `open()` returns `-ENODEV` if the key is not present (additional safety in case someone tries to access the char device without the `/dev` node).
- Synchronization for stack operations is done with `rw_semaphore` (same as Lab 4).
- A mutex protects `/dev/int_stack` create/destroy and `key_present`.

How it works

Kernel module `lab5/usb_int_stack.c` contains two independent parts:

1. **Stack implementation** (same logic as Lab 4):
    - The stack buffer (`data`) is allocated in `module_init()` and freed only in `module_exit()`.
    - `read()` implements `pop()`:
        - returns `0` bytes if stack is empty (userspace prints `NULL`)
    - `write()` implements `push(int)`:
        - returns `-ERANGE` if stack is full
    - `ioctl(INT_STACK_IOCTL_SET_SIZE)` allocates a new buffer, replaces the old one, and resets `top=0`.

2. **USB “electronic key” gate**:
    - The module registers a USB notifier (`usb_register_notify`).
    - When any device is added/removed, the notifier callback checks VID/PID:
        - if it matches `USB_KEY_VID` / `USB_KEY_PID` (Ventoy USB), it toggles `key_present`
    - On **key insert**, the module calls `device_create(...)` so `/dev/int_stack` appears.
    - On **key removal**, the module calls `device_destroy(...)` so `/dev/int_stack` disappears.
    - Only the `/dev` node is created/destroyed; the stack memory is not freed, so the stack content is preserved across unplug/replug.

Userspace wrapper

`kernel_stack` is the same CLI as Lab 4 (`set-size`, `push`, `pop`, `unwind`) but with one extra error message:

- If `/dev/int_stack` is missing (USB key not inserted), it prints:
    - `ERROR: USB key not inserted`

Quick start

Build:

```
cd lab5
make
```

Load module:

```
sudo insmod usb_int_stack.ko
```

Example workflows:

1. **Without key inserted**:

```
./kernel_stack pop
# ERROR: USB key not inserted
```

2. **Insert the USB key** (then `/dev/int_stack` appears):

```
ls -l /dev/int_stack
./kernel_stack set-size 3
./kernel_stack push 10
./kernel_stack push 20
./kernel_stack pop
# 20
```

3. **Remove the USB key** (then `/dev/int_stack` disappears, but stack is preserved):
    - Reinsert the key and continue popping values.

Cleanup:

```
sudo rmmod usb_int_stack
make clean
```